

Asymptotic

Parameter scope, coordinate stability, and preservation of evidence

A technical comparison with built-in Wolfram Language

Repository	VladimirReshetnikov/Asymptotic
Paclet version	1.8.0
Snapshot	921387e5ba1239bfda96e63e64e89bf63d9c41e6
Audit date	9 September 2026
Exclusion baseline	Nine existing reviews; 87 consolidated numbered findings

Central result. A fixed-parameter expansion cannot automatically be reused along a path on which that parameter becomes the new expansion variable. A concrete ordinary-composition example changes the true limiting value from the source-predicted finite part 1 to $1/2$. Two further findings isolate avoidable failures caused by large source translations; two concern an ignored Fourier option and the loss of already available quantitative remainder bounds.

Evidence boundary. Source inspection and independent mathematics establish the findings at the levels stated in the report. Twenty-five independent Python mathematical-model tests and five patch-emitter fixture tests pass. **No native Wolfram package execution, native benchmark, or full upstream suite was completed in this audit.** Runnable native probes, conservative prototypes, and a hash-guarded patch emitter are included.

Abstract

This is an incremental review of the Asymptotic repository, deliberately avoiding a new presentation of the findings already consolidated under its `code-review` directory. It compares implemented capabilities with current official Wolfram Language documentation, examines the ordinary and specialized expansion machinery and its operational contracts, and develops five additional findings. The strongest correctness result is a source-traced fixed-parameter capture counterexample in ordinary series composition. A second exact analysis shows that directed dyadic rounding can prevent certification of a translated linear equation at every permitted enclosure order, even when an exact root center is supplied. A related numerical-conditioning analysis explains why the ordinary inverse checker should compute in a local source coordinate. The remaining findings concern an inherited but unconsumed Fourier term-goal option and the disappearance of quantitative Zeta/Lerch tail bounds under ordinary truncation. Each finding includes its relation to the prior-review baseline, reproducible evidence, limitations, a proposed remedy, and acceptance criteria. The accompanying code separates executed independent checks from unexecuted native regression candidates and does not modify the repository.

Contents

1	Executive assessment	3
2	Scope, provenance, and novelty control	4
2.1	A pinned snapshot	4
2.2	The prior-review exclusion baseline	4
2.3	Depth of inspection	5
2.4	Evidence vocabulary	5
3	Capabilities and comparison with native Wolfram Language	5
3.1	The implemented mathematical model	5
3.2	A fair comparison	6
3.3	Comparison fixtures rather than unsupported benchmark claims	7
3.4	Positive design choices worth preserving	7
4	N01: Fixed-parameter capture in ordinary composition	8
4.1	A minimal counterexample	8
4.2	Why the existing checks do not address this example	8
4.3	The mathematical obstruction	9
4.4	A safe minimal remedy	9
4.5	A stronger, useful future contract	10
4.6	Acceptance criteria	10
5	N02: Affine cancellation is lost inside interval certification	10
5.1	The relevant implementation path	10
5.2	A linear equation with an exact supplied center	11
5.3	Why every admitted enclosure order is insufficient	11
5.4	The remedy is affine structure, not more Taylor terms	12

5.5	Performance and robustness implications	12
5.6	Patch acceptance criteria	13
6	N03: Numerical checks should use the local source coordinate	13
6.1	What is already handled correctly	13
6.2	A small inverse displacement at a large source origin	13
6.3	A coordinate-stable numerical design	14
6.4	A distinction from N02	14
6.5	Acceptance criteria and artifact scope	14
7	N04: An inherited Fourier term-goal option is never consumed	15
7.1	The declaration and the implementation disagree	15
7.2	A simple exact coefficient oracle	15
7.3	Two coherent fixes	16
8	N05: Truncation forgets quantitative evidence already available	16
8.1	The dedicated Dirichlet adapters already provide finite bounds	16
8.2	Where the information disappears	16
8.3	An exact, low-cost transport rule	17
8.4	Prototype behavior and limitations	17
8.5	A focused development opportunity	18
9	Cross-cutting implementation design	18
9.1	Treat a germ as a scoped mathematical statement	18
9.2	Keep local geometry separate from representation scale	18
9.3	Preserve evidence with a small proof algebra	19
9.4	Make inherited-option support executable	19
10	A prioritized development sequence	19
11	Validation strategy and reproducibility	20
11.1	Executed independent checks	20
11.2	Native tests supplied, not executed	21
11.3	Reproduce the independent results	21
11.4	Metamorphic tests that add coverage beyond example lists	22
12	Conclusions	22
A	Artifact inventory	22
B	Source locators for the additional findings	23

1 Executive assessment

The package’s strongest asset is not a claim that native Wolfram Language lacks asymptotics. It is its explicit representation of generalized asymptotic information: complete logarithmic blocks, selected nonstandard coordinates, exact dominant carriers, retained source equations, and distinctions between a formal residual, an asymptotic error scale, numerical evidence, and a pointwise certificate. These are useful implemented structures on top of a capable symbolic system.[1, 2, 3, 4, 5]

The existing reviews have already covered many obvious and important issues. This report does not reissue their dense-export, fractional-power, ambient-assumption, generic resource-budget, refinement-planning, release-validation, or broad roadmap recommendations. Instead, it concentrates on failures of information transport between otherwise useful components. The resulting priorities are as follows.

ID	Priority	Additional finding	Immediate response
N01	High	A fixed outer parameter can become the inner expansion variable during composition.	Reject scope capture unless a valid uniform contract or exact source replay justifies the new limit.
N02	Medium–high	Intermediate interval rounding destroys affine cancellation and can exhaust every allowed certificate precision.	Recognize exact rational affine subexpressions before rounding.
N03	Medium	Numerical checking in a huge absolute source coordinate can lose the small displacement being checked.	Solve and measure errors in a local source coordinate; reconstruct the offset only for presentation.
N04	Medium	The Fourier inverse inherits SeriesTermGoal , but its constructor never consumes it.	Implement the option or reject nondefault values explicitly.
N05	Medium	Generic truncation discards existing quantitative tail bounds, including under semantic no-op truncation.	Transport explicit bounds and their conditions with the finite part.

The classifications matter. N01 concerns the truth of the resulting asymptotic claim. N02 is *not* an unsound interval proof: outward rounding remains enclosing, but a trivially certifiable problem becomes unavailable. N03 is a numerical robustness defect, not a reported false interval certificate. N04 is an API contract omission. N05 is a loss of useful evidence, not an allegation that the finite approximation becomes wrong. These distinctions should determine both issue severity and the form of a fix.

The strategic recommendation is to strengthen three narrowly defined invariants: fixed parameters remain fixed unless explicitly transported; changing an exact source origin does not needlessly destroy local evidence; and operations preserve, weaken, or explicitly invalidate quantitative evidence rather than silently forgetting it. These recommendations are developed into concrete designs below, rather than used as a pretext to repeat the existing general architecture proposals.

2 Scope, provenance, and novelty control

2.1 A pinned snapshot

All implementation conclusions concern commit 921387e5ba1239bfda96e63e64e89bf63d9c41e6. The repository identifies the packet as version 1.8.0 and documents Wolfram Language 15.0 or later. Source references in the bibliography use the immutable commit, not a moving `main` URL. The root standalone distribution is generated from the canonical modular implementation; this review treats the canonical kernel files as the primary code evidence.[1, 2, 6]

The analysis used accessible GitHub connector reads. Local repository download attempts and native Wolfram connection attempts were unsuccessful. Thus the archive is an audit deliverable, *not* a complete repository mirror, and there is no local package-execution transcript hidden behind the independent test counts.

2.2 The prior-review exclusion baseline

The maintained `CODE_REVIEW_STATUS.md` consolidates 87 numbered findings from nine reviews. It also records the two focused fixes already present in the pinned snapshot. Those resolved items are not new findings here. Individual finding ledgers or `README` summaries were inspected for reviews 1–8; selected sections of review 9’s article and the maintained crosswalk supplied its findings. The crosswalk was used to exclude already reported issues and to identify the closest prior concerns for each proposed addition.[7, 6, 8, 9, 10, 11, 12, 13, 14, 15, 16]

This procedure is stronger than merely checking whether a new report uses different words, but it is not a claim to have re-audited every sentence and every bundled code artifact in all nine reports. The novelty assertions are specific: the counterexamples, call-path deductions, quantitative calculations, or remedies below add evidence beyond the consolidated findings. Broad subject overlap is acknowledged, not hidden.

New ID	Closest earlier topic	What is additional here
N01	C13: composition-domain candidates; X10: uniform parameters	An exact-inner, public ordinary-composition witness. The defect is parameter capture, not uncertainty crossing a domain boundary. No universal uniform-asymptotics implementation is needed to reject it safely.
N02	D04: interval-option UX; X05: stronger certificates	An exact affine counterexample with a supplied interval and supplied exact root center; a proof that all permitted precision levels fail because cancellation occurs after rounding.
N03	D05: unchecked numeric access; D06: observable conventions	Analysis of <code>InverseNumericalCheck</code> itself, rather than the intentionally lightweight numeric accessor; a concrete source-offset precision witness.
N04	D01: cutoff conventions; D02: capabilities	A particular declared option is not read at all by a particular constructor, including when its value is malformed.
N05	D02: operation support; X05: quantitative tails	The source already supplies the bounds. The new issue is their loss under a supported operation, with an elementary, implementable transport rule.

Some earlier reports include their own successful native executions. For example, review 3 records selected Wolfram Language 15.0.0 observations, while explicitly distinguishing them from a full-suite run. That historical evidence is not reclassified as this audit’s execution.[10]

2.3 Depth of inspection

Substantive reads covered the main ordinary core, series operations, composite error arithmetic, exact termination, interval certification, native special-function import, positive-real Gamma/Barnes forward normalization, Dirichlet expansions, Fourier coefficients, flat sectors, source coordinates, shared Gamma/Barnes inverse construction and power operations, numerical checks, special-function identities and real-domain contracts, exact inverse-family separation, and reciprocal-log operations. A few large files were inspected in selected windows rather than every line. The manifest records that distinction.[2, 3, 17, 18, 19, 20, 21, 22, 23, 24, 5, 25, 26, 27, 28, 29, 30]

Other modules were considered through their integration points and the documented public surface. This is not a formal verification of all mathematics in the repository, nor a complete runtime audit of all dispatch combinations. The additional findings are concentrated where the source-to-claim argument can be made explicit.

2.4 Evidence vocabulary

Source-confirmed means that a structural fact is visible in the inspected code, such as an unread option or a missing field in a newly constructed association. **Source-traced** means that a public expression has been followed through the relevant ordinary path, with an output prediction derived from that path. **Independently checked** means that the supplied Python program executed the corresponding exact arithmetic or finite-precision model. **Native probe supplied** means precisely that: executable Wolfram code is provided, but its result was not observed here.

The independent run has 25 mathematical-model tests and five patch-emitter fixture tests. The mathematical tests include 500 directed-rounding cases, 500 affine translation cases, 16 parameter-curve cases, 42 Zeta tail cases, and a separate randomized triangle-inequality check. These counts are not native package tests and should not be added to the upstream validation total.

3 Capabilities and comparison with native Wolfram Language

3.1 The implemented mathematical model

An ordinary local approach is normalized to a positive coordinate $u \rightarrow 0^+$. A generalized jet stores finitely many complete blocks

$$J(u) = \sum_{\beta \in S} u^\beta Q_\beta(\log u), \quad R(u) = \mathcal{O}\left(u^P(1 + |\log u|)^D\right), \quad (1)$$

where the ordered weights are exact and the Q_β are polynomials in the logarithmic variable. The finite set S need not lie on an ordinary integer grid. Operation representations may additionally retain an exact offset and prefactor,

$$F(x) = b(x) + A(x)(J(w(x)) + R(w(x))). \quad (2)$$

The precision of J is relative to A , whereas the reported absolute error scale includes the prefactor's magnitude. This distinction is essential when a Gamma or exponential carrier grows rapidly.[2, 3, 17]

The ordinary inverse machinery handles a shifted monomial leading term with higher-weight power-log perturbations. Specialized engines retain other useful structures: an exact Lambert core, logarithmic coordinate changes, finite flat exponential sectors, Fourier-polynomial logarithmic coefficients, and selected inverse special-function scales. These are not all freely interchangeable

coefficient fields; the implementation has deliberate supported-operation boundaries.[2, 24, 23, 22, 5, 30]

For example, the shared Gamma/Barnes inverse construction keeps exact core variables of the form

$$X_{\Gamma} = \frac{Y}{W(Y/e)}, \quad X_G = \sqrt{\frac{4Y}{W(4Y/e^3)}}. \quad (3)$$

Y denotes the respective transformed target, not the raw special-function value. Successive corrections are retained as complete polynomial blocks in reciprocal logarithms of the core. The code labels the underlying Stirling/Barnes reasoning as finite-order Poincaré asymptotics, not convergence of the complete correction series or a pointwise numeric certificate. This is useful semantic information, independently of whether a native solver can produce a related explicit expansion.[5, 26]

3.2 A fair comparison

Native **Series** is not limited to Taylor polynomials: it documents negative powers, fractional powers, some logarithmic expansions, infinity, and successive multivariate expansions. Native **Asymptotic** can infer more exotic scales. A comparison that calls every logarithm, fractional exponent, or implicit inverse a missing native feature would be inaccurate.[32, 33]

Task	Native Wolfram capability	Repository’s distinctive role or boundary
Local expansions	Series produces manipulable series objects, including selected Laurent, fractional-power and logarithmic forms.	Explicit sparse weights, complete logarithmic blocks, and a separate remainder-degree descriptor are central data rather than incidental expression structure.[32, 2]
Series reversion	InverseSeries supplies reversion, including documented ramified examples. It is not restricted to a nonzero linear coefficient in every supported case.	Generalized-weight inverse coefficients, selected special coordinates, and retained inverse-model information.[34, 2]
Implicit asymptotics	AsymptoticSolve handles equations and systems and can use non-power asymptotic scales; assumptions and conditions matter.	Specialized scalar real-branch engines with inspectable block, source and error metadata. It is not a replacement for the native system solver.[35, 2, 5]
Composition	ComposeSeries composes compatible native series representations.	Composition in the admitted power-log and reciprocal-log representations, with explicit precision transport. N01 identifies a scope condition that the ordinary path does not enforce.[36, 3, 30]
Special functions	Native symbolic and asymptotic functionality supplies broad special-function coverage.	Selected identity normalization, real-domain verification, exact carriers, and dedicated Zeta/Lerch constructions refine or supplement native output. No universal native failure is asserted.[33, 18, 27, 28, 21]

Task	Native Wolfram capability	Repository's distinctive role or boundary
Integrals, sums, equations	The native asymptotics family includes integration, summation, differential-equation and recurrence tools.	Primarily an expansion/inversion calculus, not a comprehensive reimplement of that broader family.[37, 1]
Numerical roots	FindRoot provides arbitrary-precision numerical root finding with precision and accuracy goals.	The package seeds checks from its expansions and retains the defining equation. N03 concerns the wrapper's coordinate choice, not a claim that FindRoot is unavailable or defective.[38, 31]
Pointwise evidence	A numerical solve or an asymptotic expression is not, by itself, the repository's rational-interval certificate.	Explicit continuity/domain, derivative separation and residual-enclosure checks establish a local root enclosure for supported expressions. N02 concerns avoidable enclosure loss.[4]

The practical default remains native functionality for ordinary problems that fit its representation. The package becomes especially valuable when the user needs its specific generalized coefficient calculus, retained exact core, inspectable first-omitted structure, source replay, or explicit evidence object. This is a recommendation about the demonstrated interfaces, not an unmeasured speed ranking.

3.3 Comparison fixtures rather than unsupported benchmark claims

The included native probes compare a fixed-parameter expansion with an exact diagonal substitution performed *before* expansion; ramified native reversion; native implicit inversion; and a local-coordinate numerical root calculation. The package probes address the five additional findings. The runner records a local version, source hashes, messages, timing and raw input-form results when it is executed. There are no prefilled native timing numbers.

Matching an integer order across two APIs does not necessarily match the requested information. An exclusive generalized-weight cutoff, a Taylor order, a count of complete coefficient blocks and an exponential-sector depth are different requests. This is background needed for a valid comparison, not a new finding about the already reviewed cutoff conventions. The comparison should first align the finite terms, branch, local variable and error claim; only then is a timing ratio meaningful.

3.4 Positive design choices worth preserving

Several inspected paths actively avoid common asymptotic errors. Exact Gamma/Barnes recurrences are simplified before finite tails can cancel. Half-integer modified-Bessel identities retain both exponentials instead of promoting a terminating dominant expansion to an exact function. Real-domain analysis precedes projection of native output onto a real expression. Flat inversion requires an exact zero sector rather than treating a truncated ordinary expansion as exact. Generic differentiation requires an appropriate derivative contract instead of differentiating a bare magnitude bound. These safeguards are concrete code, not merely documentation aspirations.[19, 20, 27, 28, 23, 3]

The recommendations below are designed to extend this style of conservatism to parameter scope, affine coordinates and finite quantitative evidence.

4 N01: Fixed-parameter capture in ordinary composition

Classification. High-priority correctness finding; source-traced public counterexample and independent exact checks.

4.1 A minimal counterexample

For fixed $a > 0$, let

$$F(x; a) = \frac{1}{1 + x/a}. \quad (4)$$

As $x \rightarrow 0^+$ with a fixed,

$$F(x; a) = 1 + \mathcal{O}_a(x). \quad (5)$$

The subscript is explanatory: the big- $\mathcal{O}$ constant and the neighborhood are allowed to depend on the parameter. Now construct the exact inner germ $x = a$ as $a \rightarrow 0^+$. The true composed source is

$$F(a; a) = \frac{1}{2}. \quad (6)$$

Consequently, $1 + \mathcal{O}(a)$ is false: the difference from the finite part is the nonvanishing constant $-1/2$.

The corresponding public reproduction candidate is:

```
outer = AsymptoticExpansion[1/(1 + x/a), {x, 0, 1},
  Assumptions -> a > 0];
inner = AsymptoticExpansion[a, {a, 0, 2}];
composed = SeriesCompose[outer, inner];
```

The ordinary source path predicts a finite part 1 and a first-power remainder in the inner coordinate. This is a deduction from the inspected implementation, **not a captured native output**. The native probe records the entire returned objects so that a kernel run can confirm both dispatch and metadata rather than merely compare pretty-printed expressions.[3, 2]

4.2 Why the existing checks do not address this example

The outer input is an ordinary real rational function on the indicated positive domain. The inner input is exact. Its leading coefficient is the positive constant 1, its valuation is positive, and it approaches the outer endpoint. There is no oscillatory unknown error, no complex branch choice, and no uncertainty that could cross a boundary. Both appearances of a remain positive along the diagonal.

The ordinary composition path converts the two operands to jet representations, expands the outer scale in the inner jet, composes retained coefficient blocks, and transports an outer error exponent P by the inner scale valuation α . In this example $P = \alpha = 1$. The finite outer row is just the constant 1, so the parameter that controls the discarded coefficient is not visible in that row. The path combines assumptions and substitutes the finite inner expression into the outer domain, but does not verify that an outer fixed parameter has become the new varying coordinate.[3]

This is different from the earlier C13 composition-domain candidate: exactness of the inner germ eliminates the uncertain-inner issue. A stronger domain predicate such as $a > 0$ does not imply

a uniform estimate near $a = 0$. The missing information is the quantifier scope of the error statement.

4.3 The mathematical obstruction

Proposition 4.1. *A pointwise-in-parameter remainder $R(x; a) = \mathcal{O}_a(x^P)$ does not in general imply $R(g(t); a(t)) = \mathcal{O}(g(t)^P)$ when $a(t)$ varies with t , even if g is exact, all values lie in the recorded domain, and $g(t) \rightarrow 0$.*

Proof. For (4),

$$R(x; a) = -\frac{x}{a+x}, \quad |R(x; a)| \leq \frac{x}{a} \quad (x, a > 0).$$

Thus the fixed-parameter first-order claim is valid. Set $g(t) = a(t) = t$. Then $R(t; t) = -1/2$, so $|R(t; t)|/t \rightarrow \infty$. Every point stays in the positive domain, and the inner substitution is exact. $\square$

In quantifier form, the input establishes

$$\forall a > 0 \exists C_a, \delta_a > 0 \forall x \in (0, \delta_a) : |R(x; a)| \leq C_a x^P.$$

The desired composition requires an estimate along the curve, with a constant independent of t and a neighborhood that contains that curve. Interchanging these requirements is not a formal substitution rule. The distinction between pointwise and uniform asymptotics is part of the standard meaning of such estimates.[39]

The problem is not limited to a wrong limiting constant. On $x = t^\alpha$, $a = t^\beta$,

$$R = -\frac{t^{\alpha-\beta}}{1 + t^{\alpha-\beta}}. \tag{7}$$

If $\alpha > \beta$, the error decays at order $\alpha - \beta$, not α . If $\alpha = \beta$, it is constant. If $\alpha < \beta$, the finite part 1 is not even the correct limiting value of the source. A second witness, $1/(a+x) = a^{-1} + \mathcal{O}_a(x)$, produces an error $-1/(2a)$ on the diagonal. The supplied exact checks cover these identities and a grid of power curves.

4.4 A safe minimal remedy

A minimal fix should *not* attempt to infer generic uniform asymptotics. Before composing an inexact outer germ, compare its fixed-parameter dependencies with the inner asymptotic variable. If the inner variable was a fixed outer parameter, reject the operation with a dedicated diagnostic unless a separately represented uniform contract permits the substitution.

The dependency set cannot be reconstructed solely from `Normal[outer]` or retained coefficients. In the principal witness those are independent of a . Dependencies must come from the original source, assumptions, the remainder contract, and eventually the operation recipe. A production design should record them when a germ is first constructed, before truncation can erase their visible occurrence.

The supplied `ParameterSafeCompose` is deliberately narrow. It uses a retained ordinary forward source or explicit `FixedParameters` metadata; it rejects unsupported attempts to infer scope for other inexact objects. It then delegates safe-looking cases to the existing composition routine. It is a conservative protection against this witness, not a complete proof of arbitrary composition validity and not a substitute for the existing domain/precision checks.

Exact outer expressions are a separate case. An exact identity can often be specialized without a missing big- O constant, although its branch and domain still need checking. Likewise, an exact

inner expression permits *re-expansion of the exact composed source* when the original outer source is retained. For the witness, performing $x \mapsto a$ in the source first gives $1/2$, which can be expanded safely. This is source replay, not a justified manipulation of the old truncated remainder.

4.5 A stronger, useful future contract

A genuinely reusable uniform object would retain an explicit parameter set K , a joint approach region U , and either a uniform existential estimate or a finite bound

$$|F(x; a) - J(x; a)| \leq B(x, a) \quad ((x, a) \in U). \quad (8)$$

Composition would have to prove eventual membership $(g(t), a(t)) \in U$ and compare $B(g(t), a(t))$ with the requested output scale. In the example the elementary bound x/a is sufficient to diagnose the correct loss on many curves. This is a concrete transport interface, not a proposal that every source must first receive a multivariate expansion.

There is also a useful distinction between *specializing a parameter to a fixed value* and *diagonalizing a family*. A user-facing operation should not make the latter look like an innocuous rename. An error should identify the captured parameter, the old and new varying variables, and the missing uniformity obligation.

4.6 Acceptance criteria

The minimal regression set should include the diagonal witness, the second divergent-error witness, an independent inner variable t with a genuinely fixed, an exact outer polynomial, and a parameter that occurs only in an omitted coefficient. A test that merely checks whether the finite expression contains the parameter will miss the main case. A test that checks only positivity will also miss it.

The native comparison does not assume that `ComposeSeries` must diagnose every possible later parameter substitution. Native `Series` explicitly treats distinct variables as independent. The important comparison is between expansion at fixed parameter followed by an unjustified diagonal reuse, and substitution into the exact source before expansion. The latter is the appropriate independent reference for this example.[32, 36]

5 N02: Affine cancellation is lost inside interval certification

Classification. Medium-high availability finding; exact rational derivation and executed independent rounding model. No unsound certificate is alleged.

5.1 The relevant implementation path

The certificate engine uses rational interval endpoints and directed dyadic rounding. For a nonzero rational q , `certRound` chooses a grid with a fixed number of significant bits; `certAdd` rounds after every interval addition. The generic `Plus` evaluator folds over the individually enclosed terms, starting from the zero interval. The certificate then encloses the derivative and the center residual and uses either residual containment or exact endpoint signs to establish a root.[4]

An important positive detail is already present in the same module: `certPositiveCondition` recognizes rational affine differences and evaluates their endpoint values exactly to avoid losing contact with a boundary. That shortcut is used for domain predicates, but not for the generic residual `Plus` evaluator. N02 therefore identifies an inconsistency between two uses of the same interval machinery, not an absence of all affine handling. The minimal production change could factor out and reuse a common affine-range helper.[4]

This is a reasonable basic interval design. The defect is where exact affine cancellation occurs relative to rounding. An exact constant may already be widened when added to zero, before it is combined with the variable interval whose large offset cancels that constant.

5.2 A linear equation with an exact supplied center

Let

$$A = 2^{10000} + 1, \quad f(x) = x - A, \quad y = \frac{1}{2}, \quad c = A + \frac{1}{2}, \quad I = \left[A + \frac{1}{4}, A + \frac{3}{4} \right]. \quad (9)$$

The exact derivative is 1. The exact center residual is 0. The exact root is the supplied center, strictly inside the supplied verification interval. There is no difficult transcendental tail, uncertain target, automatic-bracketing issue, or ill-conditioned mathematical inverse.

A public native reproduction candidate is:

```
A = 2^10000 + 1;
s = AsymptoticInverse[x - A, {x, A}, {y, 2}];
InverseCertificate[s, 1/2,
  "Interval" -> {A + 1/4, A + 3/4},
  "Center" -> A + 1/2,
  "EnclosureOrder" -> 2000,
  "MaxRefinements" -> 0,
  "RefineExpansion" -> False]
```

All quantities are exact. The unusually large translation is intentional: it separates a structural precision problem from a problem that might disappear after one ordinary refinement.

5.3 Why every admitted enclosure order is insufficient

The implementation restricts the enclosure order to $2 \leq n \leq 2000$ and sets

$$B = 4(n + 10) \leq 8040 \quad (10)$$

significant bits. For $q \neq 0$, the rounding grid is

$$G(q, B) = 2^{\lfloor \log_2 |q| \rfloor - B + 1}. \quad (11)$$

These are implementation facts, not inferred machine-precision behavior.[\[4\]](#)

In the center residual, the rational constant is

$$-A - \frac{1}{2} = -2^{10000} - \frac{3}{2}.$$

At the first addition to zero, its enclosure becomes

$$\left[-2^{10000} - G, -2^{10000} \right], \quad G = 2^{10000 - B + 1} > \frac{3}{2}. \quad (12)$$

Adding the exact point $c = 2^{10000} + 3/2$ gives, before any additional outward rounding,

$$\left[\frac{3}{2} - G, \frac{3}{2} \right]. \quad (13)$$

Further outward rounding cannot make this narrower. At the maximum allowed order, $G = 2^{1961}$. The residual radius divided by the derivative lower bound is therefore vastly larger than the available quarter-unit margin.

Endpoint evaluation has the same problem. Before final rounding, the left and right residual intervals are respectively

$$[5/4 - G, 5/4], \quad [7/4 - G, 7/4].$$

Both straddle zero. Thus neither the center-containment route nor the endpoint-sign route can certify the root through this rounded affine evaluation.

Proposition 5.1. *For the constant-first affine `Plus` fold in the inspected certificate evaluator, the data (9) fail the residual-containment and endpoint-sign tests for every permitted enclosure order.*

Proof. Equation (10) gives $G \geq 2^{1961}$. Equation (13) implies a residual absolute bound greater than $1/4$, whereas c is only $1/4$ from either endpoint of I . Both endpoint enclosures contain positive and negative values, so they cannot establish the required signs. The exact derivative enclosure $[1, 1]$ does not compensate for the widened residual. $\square$

The independent Python model implements the exact grid construction, rational floor/ceiling rounding, and the constant-first interval fold. It confirms the stated failure geometry at several bit counts, including 8040, without calling Wolfram Language. The public native probe remains necessary to capture the precise returned failure object and all wrapper details.

5.4 The remedy is affine structure, not more Taylor terms

An affine expression $ax + b$ with exact rational a, b has the exact interval range

$$\text{hull}\{a\ell + b, ah + b\}, \quad x \in [\ell, h]. \quad (14)$$

Compute those rational endpoints *before* rounding. For the center residual in (9), both endpoints are exactly zero. The answer remains $[0, 0]$ at any valid rounding precision.

The proposed helper recognizes rational affine trees using only constants, the source variable, `Plus`, and `Times`. It carries pairs (a, b) , combines sums exactly, and multiplies pairs only when the quadratic coefficient is zero. It neither expands an arbitrary polynomial nor calls a symbolic solver. Unsupported or nonlinear syntax falls back to the existing evaluator.

The hash-guarded patch emitter inserts this attempt into the `Plus` branch of `certEnclose`. The affine helper can consequently also resolve a translated affine argument encountered recursively inside another supported expression. It does not change the interval proof method or accept an approximate cancellation as exact. The required endpoints are still rounded outward after exact algebra.

5.5 Performance and robustness implications

Increasing an exponential/logarithmic Taylor enclosure order to resolve an affine cancellation is the wrong cost model. It simultaneously spends more work on irrelevant series tails and still encounters a fixed ceiling. The proposed structural path has cost proportional to the affine expression tree plus exact rational arithmetic on the input integers. It does not claim constant-time arithmetic independent of integer bit length.

The improvement is therefore not a speculative microbenchmark claim. The old route has an explicit precision barrier for this witness; the proposed route obtains zero residual by finite exact arithmetic. Five hundred independent randomized affine translation cases check that translating a coordinate and compensating its constant leaves the proposed enclosure unchanged.

The patch is intentionally narrow. It does not solve the general interval dependency problem, arbitrary nonlinear cancellation, or every ill-conditioned representation. Those broader topics were already within the prior-review development space. The additional result here is the exact affine failure and a small remedy with a clear proof obligation.

5.6 Patch acceptance criteria

A native validation should test positive and negative slopes, rational translations, nested factored affine expressions, exact center residuals, endpoint sign preservation, and conservative fallback for x^2 and unsupported functions. Existing certificate tests must continue to pass. The emitter itself is read-only and refuses a source blob other than the audited `InverseCertificates.wl`; it produces a diff for inspection rather than silently patching an installed package.

6 N03: Numerical checks should use the local source coordinate

Classification. Medium-priority numerical robustness finding; source-traced risk with an independently executed finite-precision witness.

6.1 What is already handled correctly

The ordinary inverse checker substitutes the exact target before numerical evaluation. It also recovers a source root for powered observables, checks the selected source side, and labels the result as numerical evidence rather than an interval certificate. Those choices are appropriate. The Gamma/Barnes checker similarly uses an exact logarithmic target transformation before evaluation.[\[31, 26\]](#)

The remaining issue is independent of exact target substitution: the ordinary checker evaluates the full source-point approximation, including its endpoint offset, at `WorkingPrecision + 10`, and asks `FindRoot` to solve for the original source variable. It later subtracts the endpoint to test the source side and compares absolute source-point values for the power-one observable.[\[31\]](#)

6.2 A small inverse displacement at a large source origin

Take

$$A = 10^{100}, \quad f(x) = (x - A) + (x - A)^2, \quad y = 10^{-3}. \quad (15)$$

Writing $u = x - A$, the positive local solution is

$$u_* = \frac{2y}{1 + \sqrt{1 + 4y}} = y - y^2 + 2y^3 - 5y^4 + \dots \quad (16)$$

It is approximately 10^{-3} , while the full source point is approximately 10^{100} .

At 60 significant decimal digits, adding a displacement of that size to 10^{100} cannot preserve it. This is a loss of roughly 103 digits of scale separation before one asks for further local accuracy. The included decimal model confirms that the full rounded sum equals the offset, even though the local displacement is strictly positive and the local equation is well resolved at high precision. It also confirms that two distinct local approximations can become the same rounded full source point.

The public native probe is:

```
A = 10^100;
s = AsymptoticInverse[(x - A) + (x - A)^2,
  {x, A}, {y, 4}];
InverseNumericalCheck[s, 1/1000, WorkingPrecision -> 50]
```

The exact native outcome may depend on numerical evaluation, messages and precision tracking: possible symptoms include failure to converge, loss of a usable source-side displacement, or a

comparison dominated by cancellation. No specific failure tag or false success is claimed as observed. The source establishes where the vulnerable operation occurs; the independent model establishes the numerical scale obstruction.

6.3 A coordinate-stable numerical design

The ordinary expansion constructor already knows the local chart. The numerical checker should retain that chart through the solve. At a finite endpoint, use $x = x_0 + \sigma u$ with $u > 0$, and solve

$$f(x_0 + \sigma u) = y \quad (17)$$

in u . Substitute the exact endpoint before approximate evaluation so that affine cancellation is available symbolically. At infinity, use the recorded reciprocal coordinate when it is numerically appropriate rather than blindly restoring a large source value first.

For a power-one finite-source observable, the absolute source-point error is exactly the local displacement error:

$$|(x_0 + \sigma u_*) - (x_0 + \sigma \tilde{u})| = |u_* - \tilde{u}|. \quad (18)$$

There is no reason to perform the two ill-scaled additions before computing this quantity. For other observables, evaluate the appropriate local observable before any unnecessary restoration of an offset.

The result object can report the local root, the exact source offset, the local-to-source transformation, and a reconstructed full source value. The full value may require substantially more precision to display a small displacement faithfully. A checker should not silently claim that a requested number of significant digits in a huge absolute point means the same number of digits in its small displacement.

Native `FindRoot` already exposes working precision, precision goals and accuracy goals. The proposed change is to use that numerical machinery on a better-conditioned coordinate representation and to select goals for the quantity actually being compared. It is not a recommendation to replace native root finding.[38]

6.4 A distinction from N02

N02 concerns exact rational interval arithmetic with an explicitly bounded significant-bit grid; the proof remains enclosing but becomes too wide. N03 concerns arbitrary-precision numerical evidence and coordinate conditioning. Their common remedy is to preserve cancellation structurally, but their correctness obligations differ. A numerical change cannot be used to justify an interval certificate, and an interval-specific affine shortcut does not automatically fix every numerical root wrapper.

The logarithmic Gamma/Barnes target transformation should be preserved. It avoids evaluating enormous special-function values unnecessarily. It does not, by itself, eliminate cancellation in a separately large affine *source* offset: the inspected checker still solves its logarithmic equation in the original source variable. That is a related test opportunity, not a separately claimed reproduced failure.[26]

6.5 Acceptance criteria and artifact scope

Compare the untranslated equation $u + u^2 = y$ with the translated equation (15), across several exact offsets and several working precisions. The meaningful invariant is local root/error agreement, not identical printed absolute coordinates. Preserve the original equation, branch checks, and explicit distinction between numerical evidence and certification.

The archive includes the precision witness and native comparison probes, but no purported complete replacement for the generic numerical checker. Correctly handling all source charts, powered observables, target transformations and domain predicates requires a coordinated implementation change. The article specifies the invariants for that change rather than disguising a restricted example solver as a production patch.

7 N04: An inherited Fourier term-goal option is never consumed

Classification. Medium-priority API contract finding; source-confirmed omission, with native probes supplied.

7.1 The declaration and the implementation disagree

The Fourier module declares its options by joining the ordinary inverse options with a frequency budget. This includes `SeriesTermGoal`. The `fourierConstruct` option extraction reads assumptions, direction, power, term and frequency budgets, input remainder, method and truncation. It never reads or validates `SeriesTermGoal`. The Fourier public interface requires an explicit exact cutoff.[22, 2]

An explicit cutoff could reasonably be documented to override a term goal. The issue here is that this specialized public interface offers no term-goal construction path at all, yet inherits the option without an explicit unsupported-option policy. An option appearing in `Options[AsymptoticFourierInverse]` is discoverable by users and tooling. Silently accepting it while ignoring both a meaningful request and a malformed value is a more concrete problem than an unusual cutoff convention.

7.2 A simple exact coefficient oracle

The zero-frequency subset supplies a particularly transparent reproduction:

```
AsymptoticFourierInverse[x + x^2, {x, 0}, {y, 4},
  SeriesTermGoal -> 1]

AsymptoticFourierInverse[x + x^2, {x, 0}, {y, 4},
  SeriesTermGoal -> "invalid"]
```

The Fourier parser admits the constant mode, and the explicit cutoff governs the construction. The source-derived prediction is that the nondefault goal does not affect the result and the malformed value is not rejected by a term-goal validator. The ordinary inverse has the three retained blocks

$$y - y^2 + 2y^3$$

below the exclusive cutoff 4. Independent polynomial arithmetic verifies that substitution into $x + x^2 - y$ cancels through cubic order and leaves $+5y^4 + \dots$. The native probes capture the actual Fourier object rather than assuming a display convention.

An oscillatory fixture such as $x + x^2 \sin(\log x)$ should also be added to production tests. Its first correction is $-y^2 \sin(\log y)$; the same option-consumption issue applies. The zero-frequency fixture is useful because it separates the option bug from Fourier coefficient complexity.

7.3 Two coherent fixes

The smallest consistent fix is to reject non-`Automatic` term goals with a specific unsupported-option message. The supplied `CheckedFourierInverse` wrapper does exactly that and otherwise delegates unchanged. It neither pretends to implement a new truncation scheme nor silently gives precedence to the explicit cutoff.

A fuller implementation could count nonzero complete source-weight/Fourier blocks, determine a boundary after the requested count, and clip against an explicit cutoff when both constraints are supplied. It would need a documented counting convention: modes within one coefficient block are not necessarily separate asymptotic orders, and exact resonance/cancellation can change the number of nonzero blocks.

The minimal fix should be preferred until that semantics is implemented. An option-consumption regression can be very small: exercise each inherited nondefault option and ensure it affects the request, is explicitly normalized, or yields a deliberate rejection. This is a targeted test of the Fourier declaration, not a repeat proposal for a repository-wide generic resource or capability redesign.

8 N05: Truncation forgets quantitative evidence already available

Classification. Medium-priority evidence-preservation finding; source-confirmed metadata loss and an exact transport theorem.

8.1 The dedicated Dirichlet adapters already provide finite bounds

For real $S > 1$, the Zeta adapter retains finitely many terms of

$$\zeta(S) = \sum_{n=1}^{\infty} n^{-S}.$$

If m is the first omitted integer, it records an explicit upper bound

$$B_m(S) = m^{-S} \left(1 + \frac{m}{S-1} \right) \quad (19)$$

and a lower bound m^{-S} for the omitted positive tail. Its positive scale is $w = e^{-S}$, so the weights are $\log n$. The metadata includes the bound, its conditions, the first omitted integer, and a stated defining-series contract.[21, 40]

These are substantive features. The bounds follow from the decreasing positive function t^{-S} :

$$m^{-S} \leq \sum_{n=m}^{\infty} n^{-S} \leq m^{-S} + \int_m^{\infty} t^{-S} dt.$$

The Lerch adapter similarly records an explicit Taylor/moment bound for fixed real parameters in its admitted geometric-series domain. This report does not propose those bounds as missing features.[21, 41]

8.2 Where the information disappears

Ordinary `SeriesTruncate` converts the input to its internal series representation and calls `seriesMake`. The latter constructs a fresh association containing the finite expression, asymptotic

remainder scale, block data, domain, recipe and related core fields. It does not copy or transport `AbsoluteRemainderBound` or `RemainderBoundConditions`.^[3]

For example:

```
s = AsymptoticExpansion[Zeta[x], {x, Infinity, Log[4]}];
r = SeriesTruncate[s, Log[5]];
```

The original finite part is $1 + 2^{-x} + 3^{-x}$, with first omitted integer 4. A larger truncation cutoff removes no retained blocks and cannot improve the old remainder. Yet the generic reconstructed object loses the quantitative fields.

This is a *semantic* no-op on the approximation, not necessarily a syntactic `SameQ` no-op on its display expression: the reconstructed expression may use powers of e^{-x} instead of n^{-x} . On the recorded real domain they represent the same finite function. The supplied probe therefore checks equality under $x > 1$, not merely identical expression trees.

The old operand may remain accessible through its recipe, so the evidence is not necessarily unrecoverable from all provenance. Nevertheless, the result no longer directly exposes the bound, and there is no implemented generic rule that tells a consumer how to recover and transport it. That is a useful capability loss under a supported operation. It should not be exaggerated into a theorem that the returned big- O estimate is false.

8.3 An exact, low-cost transport rule

Proposition 8.1. *Suppose on a domain D the source F and finite approximation A satisfy*

$$|F - A| \leq B,$$

where B is an explicit nonnegative bound. For any new finite approximation $\tilde{A}$ to the same source,

$$|F - \tilde{A}| \leq B + |A - \tilde{A}| \quad \text{on } D. \quad (20)$$

Proof. Write $F - \tilde{A} = (F - A) + (A - \tilde{A})$ and apply the triangle inequality. $\square$

Under truncation, $A - \tilde{A}$ is the known discarded finite expression. No new special-function analysis is required. If no terms are removed and equality is established on D , the bound is unchanged. If terms are removed, a conservative new bound is immediate. It may be weaker than a freshly optimized source-specific bound; it is still valid and retains information that the current generic reconstruction drops.

For Zeta, positivity can sometimes make the discarded contribution especially simple. A generic implementation should not assume positivity for all sources. Nor should it copy an old signed lower bound without analyzing the sign of the discarded expression. The prototype intentionally transports only the absolute upper bound and its original conditions.

8.4 Prototype behavior and limitations

`BoundPreservingTruncate` first calls the existing truncation routine. It then reads the explicit source bound and its conditions, computes the finite difference, optionally simplifies that difference under the retained conditions with a bounded simplification attempt, and attaches (20). If an explicit bound or its conditions are missing, it returns a diagnostic rather than treating an asymptotic scale as a numeric bound with constant one.

The wrapper assumes a well-formed source object whose quantitative contract is trustworthy. It does not independently certify the source bound, validate arbitrary hand-built associations, or

turn the resulting expression into a rational interval. Those are different obligations. It records the input bound and discarded finite expression so that the transport step itself is inspectable.

The independent tests verify the Zeta integral comparison with rational finite-sum enclosures for several integer arguments, the effect of removing terms, the no-op identity, and the general triangle inequality on randomized exact rational data. These checks validate the proposed arithmetic rule; they are not native tests of the metadata API.

8.5 A focused development opportunity

The same principle gives useful evidence-preserving operations whenever explicit finite bounds are already present. If $|F - A| \leq B_F$ and $|G - C| \leq B_G$, then

$$|(F + G) - (A + C)| \leq B_F + B_G, \quad (21)$$

$$|FG - AC| \leq |A|B_G + |C|B_F + B_FB_G. \quad (22)$$

These rules can be implemented independently of a universal transseries closure mechanism. A reciprocal additionally needs a positive lower bound separating the true denominator from zero; composition needs a domain and a Lipschitz or derivative bound. Where such conditions are unavailable, the operation should preserve an explicit unsupported-evidence status rather than silently suggest that a magnitude scale remains a finite bound.

This is a concrete incremental program because the Zeta/Lerch constructors already supply the required starting evidence. It complements, but does not duplicate, the older recommendation to derive more quantitative special-function tails.

9 Cross-cutting implementation design

9.1 Treat a germ as a scoped mathematical statement

N01 shows why an expansion object needs more than a formula and an order. A minimal semantic record distinguishes the varying coordinate, the variables held fixed, assumptions, the approach region, and the scope of its remainder assertion. These fields should be established at construction and transformed deliberately by operations.

A useful distinction is between *free symbols in the retained expression* and *dependencies of the source/error contract*. The former can disappear through truncation or cancellation. The latter must survive. Rather than forcing every operation to search a large expression history, the constructor can record a compact dependency set. This is not a proposal to redesign the entire object schema in this audit; it is the smallest additional invariant needed to block the demonstrated capture.

Two scope checks are separate. A same-variable ordinary composition can be meaningful. A distinct inner variable that was an outer fixed parameter is the dangerous case here. Renaming a dummy expansion variable should not change an answer, but renaming a parameter into the independent variable is not a dummy-variable change. Tests should encode that distinction explicitly.

9.2 Keep local geometry separate from representation scale

N02 and N03 suggest retaining a stable local chart through evidence calculations. An exact shift of origin is harmless mathematics but can be damaging when exact cancellation is delayed until after rounding. The computation should distinguish a local unknown, an exact source transformation, and a presentation coordinate.

For certificates, the smallest improvement is exact affine endpoint evaluation. For numerical evidence, solve and compare in the local variable. For user-facing output, report both local and original coordinates without pretending that their precision requirements are identical. An implementation that simply increases a global working precision treats the symptom rather than the source of the loss.

The repository already uses exact coordinate transformations in several specialized paths. The proposed program is not to introduce coordinate charts from scratch. It is to carry the appropriate chart through the ordinary evidence routines and to test affine equivariance as an explicit invariant.[24, 26, 31, 4]

9.3 Preserve evidence with a small proof algebra

N05 suggests a lightweight layer of evidence transformations. A bound can carry its conditions, the source to which it applies, and a construction tag such as “Dirichlet integral comparison” or “triangle inequality after truncation”. Operations then transform evidence only by implemented rules.

There are three honest outcomes: preserve a bound unchanged, derive a weaker bound with a recorded rule, or state that the operation cannot transport that evidence. Silently dropping fields is operationally cheap but makes downstream use fragile. Copying fields indiscriminately is worse because it can preserve an invalid claim. A small rule-specific transport layer is safer than either extreme.

This layer must not confuse an asymptotic big- O scale with a numerical upper bound. Nor can a formal residual cancellation replace a bound on the original function. The package’s existing distinctions between these evidence types should remain visible through the new transport rules.

9.4 Make inherited-option support executable

N04 can be fixed without a new public request language. Maintain an explicit list of options consumed by each specialized constructor, and reject other inherited nondefault options. A test should compare the declared option surface with the constructor’s supported policy, not merely check that a standard example still produces an answer.

When an option is normalized into another request, record the normalization. When it is irrelevant by design, reject a nondefault value or document and test a deliberate no-effect policy. A malformed value should not silently disappear. These are especially useful checks for specialized functions that build their options by joining a larger parent option set.

10 A prioritized development sequence

This sequence is intentionally limited to work made newly actionable by the present findings. It is not a restatement of the earlier repository-wide roadmap.

Stage	Change	Completion criterion
1	Block demonstrated parameter capture.	N01 and hidden-parameter regressions reject or correctly re-expand; an independent inner variable remains supported. No claim of uniformity is inferred from positivity alone.

Stage	Change	Completion criterion
2	Add the rational-affine certificate path.	Exact translated linear centers give zero residual; affine translation tests and existing certificate tests pass; unsupported trees preserve the old fallback.
3	Rework ordinary numerical evidence in the local source coordinate.	Local errors and branch checks agree across large exact source translations; full-point output does not erase local evidence.
4	Resolve Fourier option consumption.	Nondefault SeriesTermGoal either has implemented semantics or fails explicitly; malformed values cannot be ignored.
5	Preserve existing finite bounds under truncation.	Semantic no-op truncation retains the bound; term removal uses a recorded valid inequality; missing bounds are not replaced by unit-constant big- O .
6	Generalize only proven transport rules.	Addition/multiplication of explicitly bounded results carries source identity and domain conditions; reciprocal/composition require their extra separation or derivative obligations.

Stages 1, 2 and 4 have small conservative implementation surfaces. Stage 3 is broader because it interacts with source charts and powered observables. Stage 5 is small for direct truncation but needs careful source identity handling before becoming a generic evidence algebra. The staging reflects dependency and risk, not an estimated calendar schedule.

A successful patch should also resist apparently attractive shortcuts. Do not infer parameter scope from the finite part alone. Do not use approximate equality to cancel certificate constants. Do not compare rounded full source values when an exact offset cancels algebraically. Do not implement term goals by counting Fourier modes without specifying that semantics. Do not preserve an old quantitative bound after changing the finite approximation without accounting for the discarded part.

11 Validation strategy and reproducibility

11.1 Executed independent checks

The archive contains actual logs and a machine-readable result file. The executed checks are:

Check family	Result
Independent mathematical-model unit tests	25 passed
Patch-emitter fixture unit tests	5 passed
Random directed-rounding subcases	500 checked
Random affine translation subcases	500 checked
Parameter power-curve subcases	16 checked
Integer-argument Zeta tail subcases	42 checked
Native Wolfram package tests in this audit	0 run

The randomized triangle-inequality test additionally checks 200 exact rational triples. Subcases are reported separately from unit-test counts; none are marketed as package regressions.

The independent code uses only the Python standard library. The interval model uses exact `Fraction` arithmetic, including rational floor and ceiling operations. The numerical-coordinate witness uses `Decimal` with an explicitly chosen precision. No floating-point result is used to decide an interval containment assertion in the rational model.

The patch fixture tests establish that the emitter checks its anchors, refuses duplicate or missing anchors, recognizes an already inserted helper, and computes the expected Git blob hash algorithm. They do *not* establish Wolfram syntax correctness or native behavior of the emitted patch. The complete pinned source was not materialized locally for applying the emitter during this audit.

11.2 Native tests supplied, not executed

`baseline_probes.wl` records the five findings and selected native comparisons in held expressions. `run_native.wls` loads a local canonical checkout in a fresh kernel, executes bounded probes, then runs `native_regressions.wlt` against the supplied nonmutating prototypes. It records the local kernel version and source hashes. The reference commit and the actual local checkout commit are distinct fields so that a modified checkout is not falsely described as pristine.

The ten prototype regressions exercise the parameter guard and its independent-variable control, exact affine ranges and nonlinear fallback, Fourier option rejection and a supported control, and quantitative-bound transport. The baseline probes are observations rather than pass/fail tests because the current implementation is expected to exhibit the reported behavior. The full upstream suite is outside this runner's claimed scope.

The Wolfram code is native-unverified. A responsible integration requires running it on the supported kernel, inspecting any messages or dispatch differences, validating the proposed patch in the canonical implementation, and then applying the repository's normal broader tests. The article does not label the prototypes production-ready.

11.3 Reproduce the independent results

From the archive root:

```
python code/verify_independent.py
python code/test_patch_emitter.py
```

Both programs are deterministic. The first writes `evidence/independent_results.json`. The included logs were captured from the actual successful runs.

For native observations in a fresh kernel, point the runner at a local checkout:

```
# PowerShell
$env:ASYMPTOTIC_REPO = 'C:\src\Asymptotic'
wolframscript -file code/run_native.wls
```

The runner does not fetch a repository or alter it. The output file `native_observations.json` is created only by an actual native run; it is deliberately absent from the delivered evidence directory. The included `native_status.json` records the failed connection attempts and unexecuted status instead.

To emit the proposed affine certificate patch:

```
python patches/emit_affine_patch.py C:\src\Asymptotic > affine.diff
```

The emitter verifies the LF-normalized Git blob of `InverseCertificates.wl` against the inspected hash. It prints a unified diff and performs no repository write. After reviewing and applying a

compatible diff, use the repository's own standalone builder and test workflow; do not hand-edit the generated distribution as a second source of truth.

11.4 Metamorphic tests that add coverage beyond example lists

These findings suggest targeted transformations of existing test cases. Rename an independent variable while keeping parameters fixed: behavior should be equivalent. Turn a fixed parameter into the new varying variable: an inexact pointwise germ should not silently retain its old order. Translate a source origin and compensate the equation exactly: local certificate and numerical evidence should remain equivalent within their admitted coordinate scope. Truncate without removing a block: the finite function and applicable quantitative bound should survive. Change an inherited option to an invalid value: its declared policy should be exercised, not bypassed.

Such tests are particularly effective here because they perturb *semantics and metadata* while leaving the elementary coefficient mathematics unchanged. The old and new input expressions can both look simple; the error lies in the relationship between their interpretations.

12 Conclusions

The repository provides a substantial specialized calculus that complements native Wolfram Language. Its use of exact carriers, complete coefficient blocks and explicit evidence types is valuable. The present additional findings concern preserving the meaning of those objects as they move through composition, numerical checking, certification and truncation.

N01 is the most important correctness result: a fixed-parameter remainder cannot be transported along a diagonal merely because the inner series is exact and the recorded domain remains positive. N02 shows that exact rational interval arithmetic still benefits critically from recognizing affine structure before rounding. N03 extends the coordinate lesson to numerical evidence, where a small local displacement should not be recovered by subtracting two huge rounded source points. N04 and N05 are smaller but concrete consistency problems: an advertised option should be consumed, and an available finite bound should not disappear under a semantic no-op.

The supplied remedies are intentionally uneven in scope. Three are conservative nonmutating wrappers; one is a small hash-guarded certificate patch proposal; one is a detailed numerical redesign specification. The independent mathematics and fixture tests ran successfully, while native verification remains explicitly pending. This separation makes the report useful for development without overstating what was executed or repeating the repository's existing review inventory.

A Artifact inventory

Path	Purpose
<code>article.tex</code> , <code>article.pdf</code>	Editable article and compiled document.
<code>code/verify_independent.py</code>	Executed exact/decimal models and deterministic unit tests.
<code>code/test_patch_emitter.py</code>	Executed fixture tests of diff-generation machinery.
<code>code/AuditExtensions.wl</code>	Nonmutating conservative composition, truncation and option wrappers; rational-affine range helper.
<code>code/native_regressions.wlt</code>	Ten native-unexecuted prototype regressions.

Path	Purpose
<code>code/baseline_probes.wl</code>	Source-derived public reproductions and native comparison fixtures.
<code>code/run_native.wls</code>	Fresh-kernel local-checkout runner and evidence exporter.
<code>patches/affine_helper.wl</code>	Proposed private rational-affine certificate helper.
<code>patches/emit_affine_patch.py</code>	Read-only pinned-blob diff emitter.
<code>evidence/findings.json</code>	Finding classifications and implementation status.
<code>evidence/novelty_crosswalk.csv</code>	Closest prior findings and the specific new contribution.
<code>evidence/source_manifest.json</code>	Pinned source references, available blob hashes and inspection limits.
<code>evidence/independent_results.json</code>	Actual independent test summary.
<code>evidence/*test_log.txt</code>	Actual successful independent and fixture test logs.
<code>evidence/native_status.json</code>	Explicit record that native work was not executed.
<code>README.md</code> , <code>build.sh</code>	Reproduction and PDF build instructions.
<code>SHA256SUMS.txt</code>	Integrity manifest for the deliverable files.

B Source locators for the additional findings

ID	Primary source	Decisive locations
N01	<code>Kernel/SeriesOperations.wl</code>	<code>SeriesCompose</code> , <code>seriesJetApply</code> , <code>seriesData</code> ; composition of retained rows and pure transported outer remainder, without a fixed-parameter capture check.
N02	<code>Kernel/InverseCertificates.wl</code>	Lines 8–24: rounding/addition; lines 99 onward: structural evaluator; lines 229 onward: residual certificate; lines 289 onward: options and enclosure-order cap. Function names are the stable locators.
N03	<code>Kernel/NumericalInverseChecks.wl</code>	<code>numericalInverseEvidence</code> : full-expression numerical seed, original-variable <code>FindRoot</code> , endpoint subtraction and absolute-point comparison.
N04	<code>Kernel/FourierCoefficients.wl</code>	Inherited options and <code>fourierConstruct</code> 's option extraction and explicit-cutoff construction.
N05	<code>Kernel/DirichletSpecialFunctions.wl</code> ; <code>Kernel/SeriesOperations.wl</code>	<code>dirichletZetaForward</code> and <code>dirichletLerchForward</code> attach bounds; <code>SeriesTruncate</code> / <code>seriesMake</code> reconstruct the result without them.

All `Kernel/` paths in this appendix are relative to `AsymptoticInverse/`. Line windows are aids to navigation at the pinned commit; the named functions and immutable URLs in the references are authoritative locators.

References

- [1] VladimirReshetnikov/Asymptotic. *Repository README*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/README.md>
- [2] VladimirReshetnikov/Asymptotic. *Ordinary expansion and inverse core*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/AsymptoticInverse.wl>
- [3] VladimirReshetnikov/Asymptotic. *Series representations and operations*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SeriesOperations.wl>
- [4] VladimirReshetnikov/Asymptotic. *Rational residual certificates and source-domain predicates*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/InverseCertificates.wl>
- [5] VladimirReshetnikov/Asymptotic. *Shared Gamma/Barnes inverse construction*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/GammaInverse.wl>
- [6] VladimirReshetnikov/Asymptotic. *Code-review index*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/README.md>
- [7] VladimirReshetnikov/Asymptotic. *Consolidated code-review status and crosswalk*. Pinned commit 921387e5ba12.
https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/docs/development/CODE_REVIEW_STATUS.md
- [8] VladimirReshetnikov/Asymptotic. *Review 1: finding ledger*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-1/evidence/findings.csv>
- [9] VladimirReshetnikov/Asymptotic. *Review 2: scope, findings and execution record*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-2/README.md>
- [10] VladimirReshetnikov/Asymptotic. *Review 3: scope, findings and native execution record*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-3/README.md>
- [11] VladimirReshetnikov/Asymptotic. *Review 4: finding ledger*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-4/evidence/findings.json>
- [12] VladimirReshetnikov/Asymptotic. *Review 5: finding ledger*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-5/evidence/findings.json>

- [13] VladimirReshetnikov/Asymptotic. *Review 6: finding ledger*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-6/evidence/findings.csv>
- [14] VladimirReshetnikov/Asymptotic. *Review 7: finding ledger*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-7/evidence/findings.csv>
- [15] VladimirReshetnikov/Asymptotic. *Review 8: finding ledger*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-8/evidence/findings.csv>
- [16] VladimirReshetnikov/Asymptotic. *Review 9: article, selected sections*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-9/article.tex>
- [17] VladimirReshetnikov/Asymptotic. *Composite error arithmetic*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SeriesEnvelopeArithmetic.wl>
- [18] VladimirReshetnikov/Asymptotic. *Native special-function import*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/NativeSpecialFunctions.wl>
- [19] VladimirReshetnikov/Asymptotic. *Forward Gamma normalization*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/GammaForward.wl>
- [20] VladimirReshetnikov/Asymptotic. *Forward Barnes G normalization*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/BarnesForward.wl>
- [21] VladimirReshetnikov/Asymptotic. *Zeta and Lerch defining-series adapters*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/DirichletSpecialFunctions.wl>
- [22] VladimirReshetnikov/Asymptotic. *Fourier coefficient inverse engine*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/FourierCoefficients.wl>
- [23] VladimirReshetnikov/Asymptotic. *Finite flat-sector inverse engine*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/FlatSectors.wl>
- [24] VladimirReshetnikov/Asymptotic. *Source-coordinate transformations*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SourceCoordinates.wl>

- [25] VladimirReshetnikov/Asymptotic. *Gamma/Barnes inverse observable operations*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/GammaInverseOperations.wl>
- [26] VladimirReshetnikov/Asymptotic. *Gamma/Barnes numerical and formal inverse checks*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/GammaInverseChecks.wl>
- [27] VladimirReshetnikov/Asymptotic. *Exact special-function identities*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SpecialFunctionIdentities.wl>
- [28] VladimirReshetnikov/Asymptotic. *Sufficient real-domain contracts*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SpecialFunctionRealDomain.wl>
- [29] VladimirReshetnikov/Asymptotic. *Exact inverse-family reduction*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/InverseFunctionFamilies.wl>
- [30] VladimirReshetnikov/Asymptotic. *Reciprocal-log composition and differentiation*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/ReciprocalLogOperations.wl>
- [31] VladimirReshetnikov/Asymptotic. *Ordinary inverse numerical evidence*. Pinned commit 921387e5ba12.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/NumericalInverseChecks.wl>
- [32] Wolfram Research. *Series*. Official documentation accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/Series.html>
- [33] Wolfram Research. *Asymptotic*. Official documentation accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/Asymptotic.html>
- [34] Wolfram Research. *InverseSeries*. Official documentation accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/InverseSeries.html>
- [35] Wolfram Research. *AsymptoticSolve*. Official documentation accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/AsymptoticSolve.html>
- [36] Wolfram Research. *ComposeSeries*. Official documentation accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/ComposeSeries.html>
- [37] Wolfram Research. *Asymptotics guide*. Official documentation accessed 9 September 2026.
<https://reference.wolfram.com/language/guide/Asymptotics.html>
- [38] Wolfram Research. *FindRoot*. Official documentation accessed 9 September 2026.
<https://reference.wolfram.com/language/ref/FindRoot.html>

- [39] NIST Digital Library of Mathematical Functions. *Section 2.1: Definitions and elementary properties*. Official documentation accessed 9 September 2026.
<https://dlmf.nist.gov/2.1>
- [40] NIST Digital Library of Mathematical Functions. *Section 25.2: Riemann zeta function, definition and expansions*. Official documentation accessed 9 September 2026.
<https://dlmf.nist.gov/25.2>
- [41] NIST Digital Library of Mathematical Functions. *Section 25.14: Lerch's transcendent*. Official documentation accessed 9 September 2026.
<https://dlmf.nist.gov/25.14>